Shared by kara. This is a copy of a chat between Claude and kara. Content may include unverified or unsafe content that does not represent the views of Anthropic. Shared snapshot may contain attachments and data not displayed here.

Report

Files hidden when shared

pelajari ini dulu

Sudah saya baca ketiganya. Ini ringkasan pemahaman saya, plus beberapa temuan yang menurut saya perlu diperhatikan sebelum coding dimulai.

Gambaran proyek

Proyek BERT-from-Scratch Kelompok 12 dengan task **MLM murni** di **WikiText-2**. Tokenizer word-level kustom dengan `vocab_size=30000` dan `max_seq_length=128`. Modelnya encoder 6 layer, hidden 256, 8 head, FFN 1024, sekitar 20 juta parameter. Training memakai AdamW, lr 5e-4, batch 32, 10 epoch, AMP FP16. Kontrak antar-modul: `input_ids`, `attention_mask`, `token_type_ids`, `labels` masuk ke model, dan `logits (B,L,V)` serta `loss` keluar, dengan `ignore_index=-100`. ID special token 0–4 terkunci.

Ketiga dokumen secara umum konsisten satu sama lain pada angka-angka utama (arsitektur, training, special token, masking 80/10/10).

Temuan yang perlu dicermati

1. **Hardware masih asumsi.** SPEC sendiri menyatakan bagian `[HARDWARE]` kosong dan semua angka diasumsikan T4. Ini perlu dikonfirmasi di rapat.
2. **Output `BERT.forward()` tidak ada di SPEC.** SYSTEM_PROMPT menyebut `MLMOutput(logits, loss)` yang mendukung akses atribut, tuple unpacking, dan `out["logits"]`. Tapi SPEC §4 hanya menyebut `logits` dan `loss` tanpa mendefinisikan tipe pengembaliannya. Karena ini menyangkut interface lintas modul (kategori b), sebaiknya dimasukkan ke SPEC agar kontraknya satu sumber.
3. **Path/struktur file berbeda.** SPEC menyebut `data.py`, `model.py`, `train.py` datar, sedangkan SYSTEM_PROMPT dan `config.yaml` memakai `data/tokenizer.py`, `data/data.py`, `model/model.py`, `training/train.py`, `training/metrics.py`, `training/utils.py`. SPEC §6 juga tidak mencakup `tokenizer.py`, `metrics.py`, `utils.py`. Perlu disinkronkan.

4. **Detail masking yang ambigu.** Kalimat di SYSTEM_PROMPT "sisanya di `labels` wajib bernilai `-100`" cukup jelas, tapi SPEC menyebut posisi yang dipilih tapi dibiarkan tetap masuk `labels`. Itu benar dan sesuai BERT asli, hanya perlu dipastikan implementor tidak keliru men-set `-100` untuk yang 10% "dibiarkan".
5. **`vocab_size=30000` vs ukuran WikiText-2.** WikiText-2 hanya punya sekitar 33 ribu tipe kata unik secara total. Dengan 30.000 sudah mencakup hampir semuanya, jadi OOV kemungkinan kecil, tapi banyak entri vocab akan sangat jarang muncul sehingga embedding-nya hampir tidak terlatih. Perlu dicek lewat statistik frekuensi, dan SPEC memang sudah menandai angka ini "final setelah analisis".
6. **Kalimat di SPEC soal "~36k baris train".** Itu benar untuk baris di HuggingFace (banyak yang kosong atau berupa header). Pembuatan sequence 128 token sebaiknya dilakukan dengan mengelompokkan/memotong teks, bukan per baris, kalau tidak banyak sequence akan sangat pendek dan didominasi padding.
7. **`layer_norm_eps: 1e-12` dan FP16.** Nilai 1e-12 di bawah rentang representable FP16 (sekitar 6e-8 untuk subnormal). LayerNorm di dalam autocast biasanya dijalankan dalam FP32 oleh PyTorch, jadi umumnya aman, tapi ini layak diuji, mengingat prompt verifikasi sudah menyorot risiko NaN pada AMP.
8. **Estimasi parameter.** Embedding sekitar 7.7M, encoder sekitar 4.8M, head sekitar 7.7M, jadi total sekitar 20M. Angkanya masuk akal.

Apa selanjutnya?

Saya siap membantu dengan salah satu dari ini, tergantung peran Anda:

- Menulis `model/model.py` (`MultiHeadAttention`, `EncoderLayer`, `BERTEncoder`, `MLMHead`, `BERT`, `MLMOutput`)
- Mereview dan merapikan SPEC (poin 2 dan 3 di atas) sebagai usulan perubahan kategori (b)
- Menyusun checklist rapat untuk hardware dan hal-hal yang tertunda
- Hal lain yang Anda butuhkan

Mau mulai dari yang mana?

Dari SYSTEM_PROMPT.md yang tadi kulampirkan, lalu patuhi Master System Prompt
di §1 selama sesi ini (aturan from scratch, kontrak SPEC, shape tensor, dan peringatan jika ada perubahan SPEC).

KONTEKS SESI INI
- Anggota & Peran: Bagas — Data Pipeline
- Hardware Target: [isi: laptop lokal CPU / Google Colab T4]
- File yang Dikerjakan: data/tokenizer.py
- Tujuan Sesi Ini: Di data/tokenizer.py: bangun word-level tokenizer dari nol (build_tokenizer, encode, decode, simpan/muat vocab JSON di

data/processed/vocab.json) dengan ID 0–4 dikunci untuk [PAD], [UNK], [CLS], [SEP], [MASK] dan vocab_size=30000. Sertakan perhitungan rasio OOV ([UNK]) untuk dilaporkan ke tim.

Konteks data: data bersih sudah ada di data/processed/ (train_clean.txt, validation_clean.txt, test_clean.txt), satu paragraf per baris, sudah lowercase, tanda baca masih terpisah spasi sehingga cukup .split(). Vocab dibangun hanya dari train. Baca semua angka dari config.yaml (terlampir).

Jelaskan alur logikanya singkat dulu, lalu beri kodenya.

Show more

Hardware: tokenizer ini murni Python dan jalan di CPU, jadi targetnya tidak memengaruhi kode. Kolom itu tetap perlu diisi di rapat untuk SPEC.

Alur logika

1. **Config.** Semua angka (`vocab_size` , ID special token, `max_seq_length` , path) dibaca dari `config.yaml` . ID 0–4 divalidasi sama dengan yang terkunci, dan program berhenti kalau tidak cocok.
2. `build_tokenizer` .
   - Hitung frekuensi kata dari `train_clean.txt` saja (`.split()`).
   - Ambil `vocab_size - 5 = 29.995` kata terbanyak, urut frekuensi turun, lalu alfabet untuk tie-break supaya deterministik.
   - ID kata mulai dari 5.
   - Jika kata unik di train kurang dari 29.995, vocab aktif lebih kecil dari 30.000.
3. `encode` .
   - `text.split()` lalu lookup, kata yang tidak ada jadi `[UNK]=1` .
   - Opsional ditambah `[CLS]` / `[SEP]` , dipotong ke `max_length` , dan dipadding dengan `[PAD]=0` .
   - Output berupa `list[int]` . Pembuatan tensor dan masking tetap tugas `data.py` .
4. `decode` . ID kembali ke kata, special token bisa di-skip.
5. **Simpan/muat.** Format JSON ke `data/processed/vocab.json` . Saat dimuat, ID special token divalidasi lagi.
6. **Rasio OOV.** Dihitung untuk train, validation, dan test, per token dan per tipe. Hasil dicetak dan disimpan ke `outputs/logs/oov_report.json` .

Dua hal untuk Bagas:

- Ukuran vocab aktif disimpan di `len(tokenizer)` . Pakai itu, bukan konstanta 30000, saat memilih ID acak 10% di `data.py` (`randint(5, len(tokenizer))`). Kalau tidak, token acak bisa menunjuk baris embedding yang tidak pernah dipakai.
- Nama file `train_clean.txt` dan seterusnya tidak ada di `config.yaml` , jadi saya jadikan konstanta di modul. Ini bukan perubahan SPEC.

Ran a command, shared files